

Outlier Detection with the Percentile Method and Winsorization

Contents

1. Overview	2
2. The percentile rule	3
2.1 What a percentile says	3
2.2 From percentiles to limits	4
3. Trimming and capping (winsorization)	5
3.1 Variants of capping	6
4. The height and weight data	6
5. Finding the limits and the outliers	7
6. Trimming in code	8
7. Capping in code (winsorization)	9
8. Choosing the cut-offs	11
9. Learning the limits on the training set	12
10. Strengths and limits of the percentile method	12
11. Comparing the three detection rules	13
12. Summary	15
13. Sources	15
14. Key terms	15

Where this fits: Pipeline map step 4 (Clean). See the [Course map](#).



- **Builds on:** Outliers ([Note 41](#)).
- **Compare with:** Z-score outlier method ([Note 42](#)); IQR outlier method ([Note 43](#)).

1. Overview

Key point: We pick a low and a high percentile, such as the 1st and the 99th; every value below the first or above the second is an outlier, and we trim those rows or cap the values at the limits.

Note 41 listed three rules for detecting outliers. Note 42 covered the z-score rule for normal features and Note 43 the IQR rule for skewed ones. This Note covers the third and simplest rule: the **percentile method** (G-1481), which works on a **feature** (G-772) (an input variable, one column of the data table) of any shape. Each **observation** (G-1374) is one record (one row), here one person.

Figure 1 shows the whole method, in three steps:

1. choose two cut-offs;
2. turn them into a lower and an upper limit;
3. treat the values outside them by trimming or capping.

Capping with percentile limits has its own name: **winsorization** (G-2123).

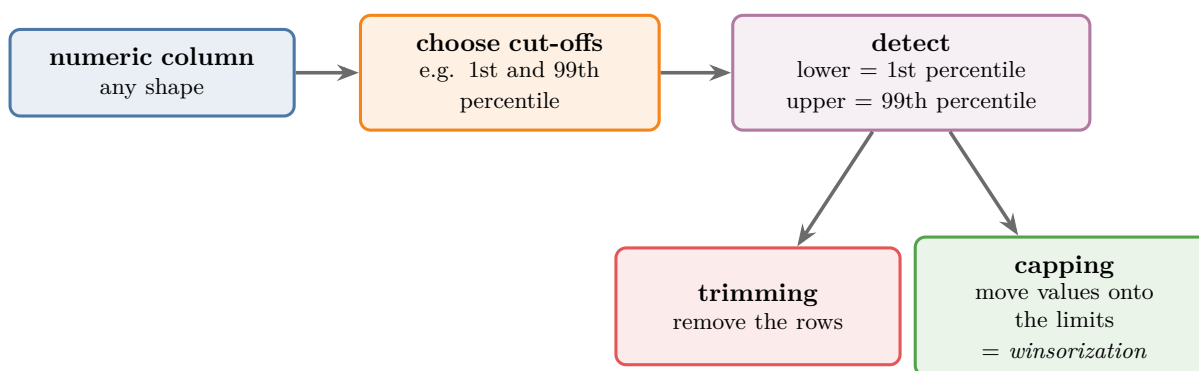


Figure 1: The percentile method: choose two percentiles as limits, then trim or cap (winsorize)

2. The percentile rule

Key point: Everything below the 1st percentile or above the 99th percentile is an outlier; the two cut-offs are our choice.

2.1 What a percentile says

Key point: A percentile is a count: the share of the values that lie below a given value.

A percentile tells where a value stands among all the others. A student whose exam score is at the 50th percentile has half the class behind them; a student at the 99th percentile has 99 percent of the class behind them. The highest score sits at the top, the 100th percentile.

The p -th **percentile** (G-1483) is the value that $p\%$ of the feature's values lie below, as the [understanding your data Note](#) (section 7.2) explains. A percentile uses only the order of the values, never their shape. The question can also be asked the other way round: the percentile at which a given value falls is that value's **percentile rank** (G-1482).

Figure 2 finds a percentile rank by counting, on the heights of 14 people from the data of Section 4 (every 700th person, rounded to whole inches). Watch the dots light up in two steps:

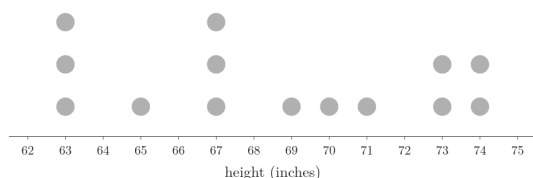
1. **In words:** count the values below the one we ask about, and divide by the number of values.

2. **Formula:**

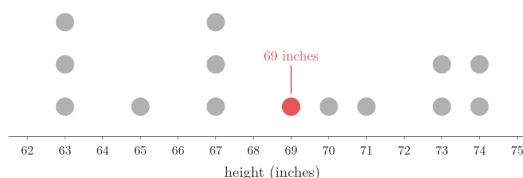
$$\text{percentile rank} = \frac{\text{values below}}{\text{all values}} \times 100$$

3. **Example:** the sorted heights are 63, 63, 63, 65, 67, 67, 67, 69, 70, 71, 73, 73, 74, 74. Seven of the 14 are below 69, so $7/14 \times 100 = 50$: a height of 69 inches is at the 50th percentile.
4. **The other rule:** some textbooks count the values *at or below*. Then the person at 69 counts too: $8/14 \times 100 = 57$, the 57th percentile.

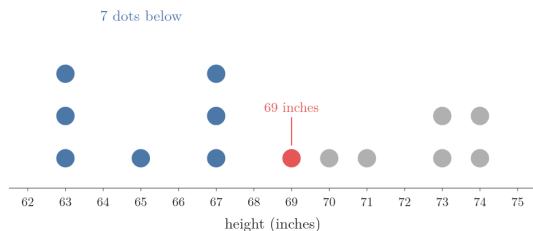
1. the heights of 14 people, one dot each



2. at which percentile is the person who is 69 inches tall?



3. count the dots below 69: 7 of 14 = 50% → 50th percentile



4. count the dots at or below 69: 8 of 14 = 57% → 57th percentile

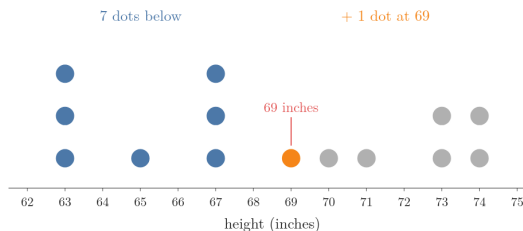


Figure 2: A percentile by counting on 14 heights: 7 dots lie below 69 inches (50th percentile); counting the dot at 69 as well gives 8 of 14 (57th percentile) (idea after Khan Academy, “Calculating percentile”)

Both rules are in use, and on small data they give different answers. This Note uses the first one, “below”; pandas’ `quantile` also interpolates between neighbouring values (Extra at the end of this section). The 10,000 heights of Section 4 are all different, so there the two counting rules differ by one person out of 10,000.

2.2 From percentiles to limits

Key point: Choose a low and a high percentile; the values there are the two limits.

The method has a single step: cut off a small slice at each end of the feature, like trimming the crusts off both ends of a loaf. Everything beyond the cut is an **outlier** (G-1420). The rule treats both ends at once.

The usual **cut-offs** (G-526) are the 1st and the 99th percentile. Other common pairs are:

- 5th and 95th: cuts more, 5% at each end;
- 0.5th and 99.5th: cuts less, 0.5% at each end.

The limits, step by step:

1. **In words:** the lower limit is the value that 1% of the values lie below; the upper limit is the value that 99% lies below.
2. **Formula:** with P_p the p -th percentile,

$$\text{lower} = P_1, \quad \text{upper} = P_{99}$$

3. **Example:** the 10,000 heights of Section 4, sorted from small to large. 1% of 10,000 is 100, so the lower limit sits between the 100th smallest height (58.126) and the 101st (58.134). pandas places it at 58.134; in the same way the upper limit sits between the 100th and 101st largest heights, at 74.786.

Figure 3 lines up all 10,000 heights from shortest to tallest. Watch where the two dashed limits cross the curve: exactly at rank 100 and rank 9,900, so the red stretches at each end hold 100 people each.

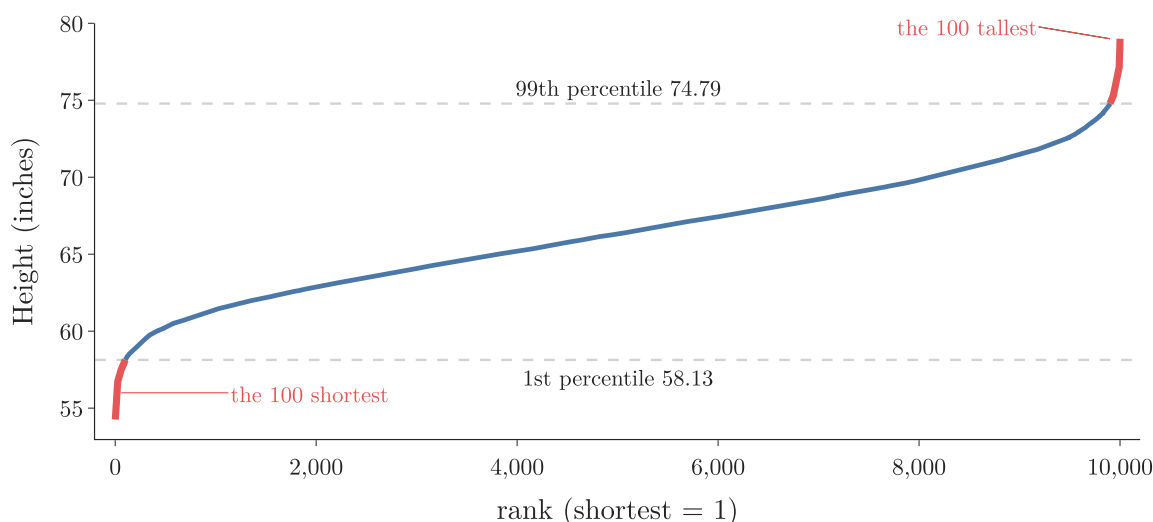


Figure 3: The 10,000 heights sorted by rank; the 1st and 99th percentiles are the heights at ranks 100 and 9,900, and the 100 shortest and 100 tallest lie beyond them

Extra: “Between” needs a rule, because 1% of the values rarely ends exactly on one value. pandas uses **linear interpolation** (G-1092) by default: it computes the position $(n - 1) \times p = 9,999 \times 0.01 = 99.99$ in the sorted column (counting from 0) and goes 99% of the way from the value at position 99 to the value at position 100. Other tools use other rules, so their limits can differ slightly (pandas docs, `Series.quantile`; Hyndman and Fan 1996 compare nine such rules).

3. Trimming and capping (winsorization)

Key point: Trimming removes the observations beyond the percentile limits; capping replaces each such value with the limit, and capping with percentile limits is called winsorization.

Trimming (G-2019), which removes the rows, and **capping** (G-345), which moves each value onto the limit it crossed, work as in the [outliers Note](#) (section seven, ways to treat outliers), with the percentile limits as the limits.

When the capping limits are percentiles, the technique is called **winsorization**, after the statistician who proposed it. Trimming with percentiles has no special name; it is simply trimming.

Extra: The name honours the statistician Charles P. Winsor, a colleague of John Tukey, who introduced the box plot (Note 20). The two worked on these methods together (Hastings et al. 1947), and Tukey credited Winsor with converting him to statistics (Brillinger 2002, §6.3).

3.1 Variants of capping

Key point: Some people cap at the limit plus or minus a small step instead of the limit itself; neither choice is fixed, so we try both and keep what works.

A few implementations do not replace an outlier with the limit itself:

- the upper outliers become **upper limit** + 1;
- the lower outliers become **lower limit** - 1.

The idea is to keep the capped values slightly apart from the normal ones. There is no fixed rule here. Outlier handling is practical and experimental: we try the options and keep the one that gives the best model.

4. The height and weight data

Key point: The data has 10,000 people with their gender, height and weight; height is close to normal and its box plot shows outliers on both sides.

The data is a table of 10,000 people, 5,000 men and 5,000 women, published on Kaggle.

Gender	Height	Weight
Male	73.85	241.89
Male	68.78	162.31
Male	74.11	212.74
Male	71.73	220.04

- **Gender:** male or female. We do not use it here.
- **Height:** in inches.
- **Weight:** in pounds.

We work on **Height**. **Weight** has almost no outliers by the box-plot rule (1 value beyond the fences), so it shows little; the same steps work on it. **Gender** and **Weight** are not used, so there is no **target** (G-1949) (an output to predict) here: outlier handling is a cleaning step done before any model.

The summary numbers of **Height**:

Count	Mean	Std	Min	25%	50%	75%	Max
10,000	66.37	3.85	54.26	63.51	66.32	69.17	79.00

Figure 4 shows the histogram: almost a bell (skewness 0.05). The box plot above it shows a few dots on both sides, so the feature does hold extreme values.

Extra: One inch is 2.54 cm. The heights run from 54.26 inches (138 cm) to 79.00 inches (201 cm), with a mean of 66.37 inches (169 cm).

Python: Loading the data and looking at Height.

```
import pandas as pd
```

```
df = pd.read_csv("data/weight-height.csv")
```

```
df.shape # (10000, 3)
```

```
df["Height"].describe() # the table above
```

The Notebook draws the histogram and box plot with Plotly.

5. Finding the limits and the outliers

Key point: The 1st percentile of height is 58.13 and the 99th is 74.79; 100 heights lie below and 100 above, so 200 people are outliers.

The two limits of Section 2:

- **lower limit:** 58.13 inches (1st percentile);
- **upper limit:** 74.79 inches (99th percentile).

Figure 4 marks them as dashed lines. Exactly 100 heights lie below the lower limit (54.26 to 58.13) and 100 above the upper limit (74.79 to 79.00), in red.

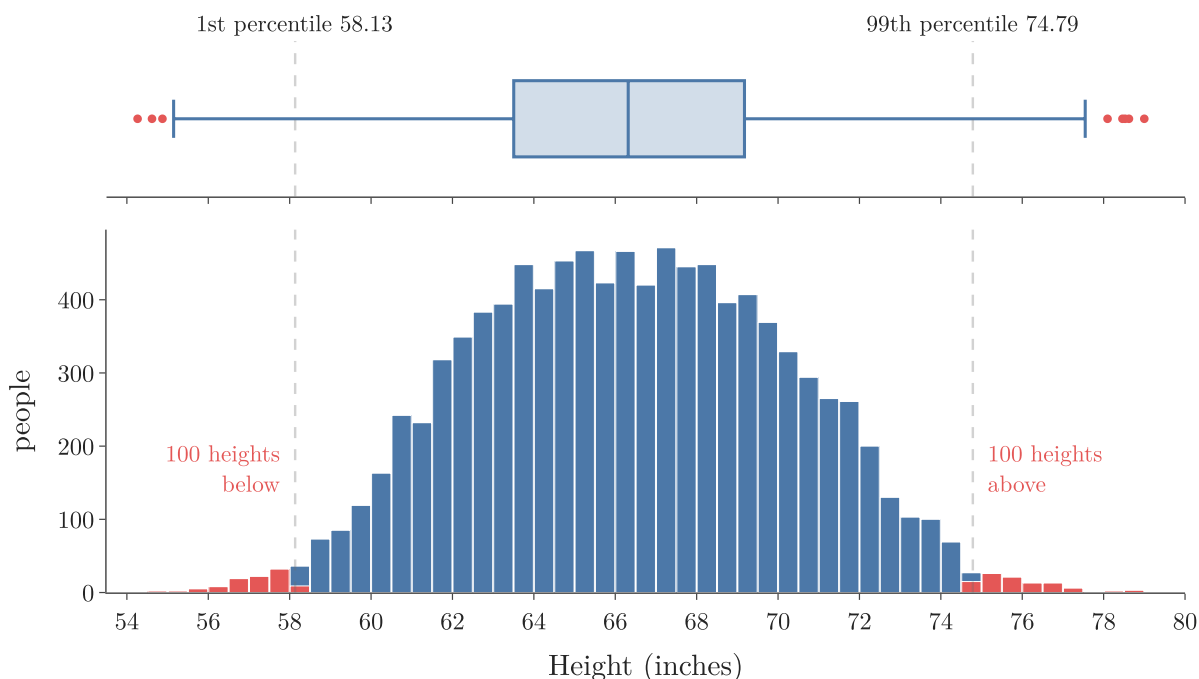


Figure 4: Height of 10,000 people: box plot above, histogram below, with the 1st and 99th percentiles dashed and the 200 values outside them in red

The percentile limits sit well inside the box-plot whiskers. The box plot flags only the 8 most extreme dots; the percentile rule flags 200 values. Section 10 comes back to this difference.

Python: The limits and the outliers.

```
# quantile takes a fraction: 0.99 = 99th percentile
upper_limit = df["Height"].quantile(0.99) # 74.7858
lower_limit = df["Height"].quantile(0.01) # 58.1344

(df["Height"] > upper_limit).sum() # 100
(df["Height"] < lower_limit).sum() # 100
```

6. Trimming in code

Key point: Keeping only the observations between 58.13 and 74.79 removes 200 and leaves 9,800.

Trimming is a filter: keep the rows whose height lies between the two limits.

Python: Trimming.

```
# keep lower <= height <= upper
new_df = df[df["Height"].between(lower_limit, upper_limit)]
```

```
new_df.shape      # (9800, 3)
```

Extra: Use the variables `upper_limit` and `lower_limit`, not rounded numbers typed by hand. Filtering with 74.78 and 58.13 keeps only 9,799 rows. One person is 74.7857 inches tall: inside the true limit of 74.7858, but above the rounded 74.78, so that row is wrongly removed.

The summary numbers before and after:

	Original	Trimmed
Rows	10,000	9,800
Mean	66.37	66.36
Standard deviation	3.85	3.65
Minimum	54.26	58.13
Median	66.32	66.32
Maximum	79.00	74.79

The mean and median stay almost the same. The standard deviation drops a little, the minimum rises and the maximum falls. Figure 5 (middle row) shows the same bell with its two thin tails cut off, and a box plot with no dots left.

7. Capping in code (winsorization)

Key point: Each height above 74.79 becomes 74.79 and each height below 58.13 becomes 58.13; all 10,000 rows stay.

Capping goes through the feature value by value, as in Notes 42 and 43:

- above the upper limit: replace it with the upper limit;
- below the lower limit: replace it with the lower limit;
- otherwise: leave it as it is.

NumPy's `np.where` (G-124), called as `np.where(condition, value if true, value if false)`, does this for the whole column; two conditions need two calls, one inside the other.

Python: Winsorization with `np.where`.

```
import numpy as np
```

```
capped = df.copy()
h = capped["Height"]
capped["Height"] = np.where(
    h > upper_limit,
    upper_limit,
    np.where(h < lower_limit, lower_limit, h))
```

```
capped.shape      # (10000, 3): no row lost
df.copy() keeps the original df unchanged for the comparison below.
```

The result, next to the original and the trimmed column:

	Original	Trimmed	Capped
Rows	10,000	9,800	10,000
Mean	66.37	66.36	66.37
Standard deviation	3.85	3.65	3.80
Minimum	54.26	58.13	58.13
Maximum	79.00	74.79	74.79

Only the minimum and maximum change much: they are now exactly the limits. Figure 5 (bottom row) shows the 100 low values piled onto 58.13 and the 100 high values onto 74.79, in orange. The histogram rises a little at both ends, and the box plot has no dots.

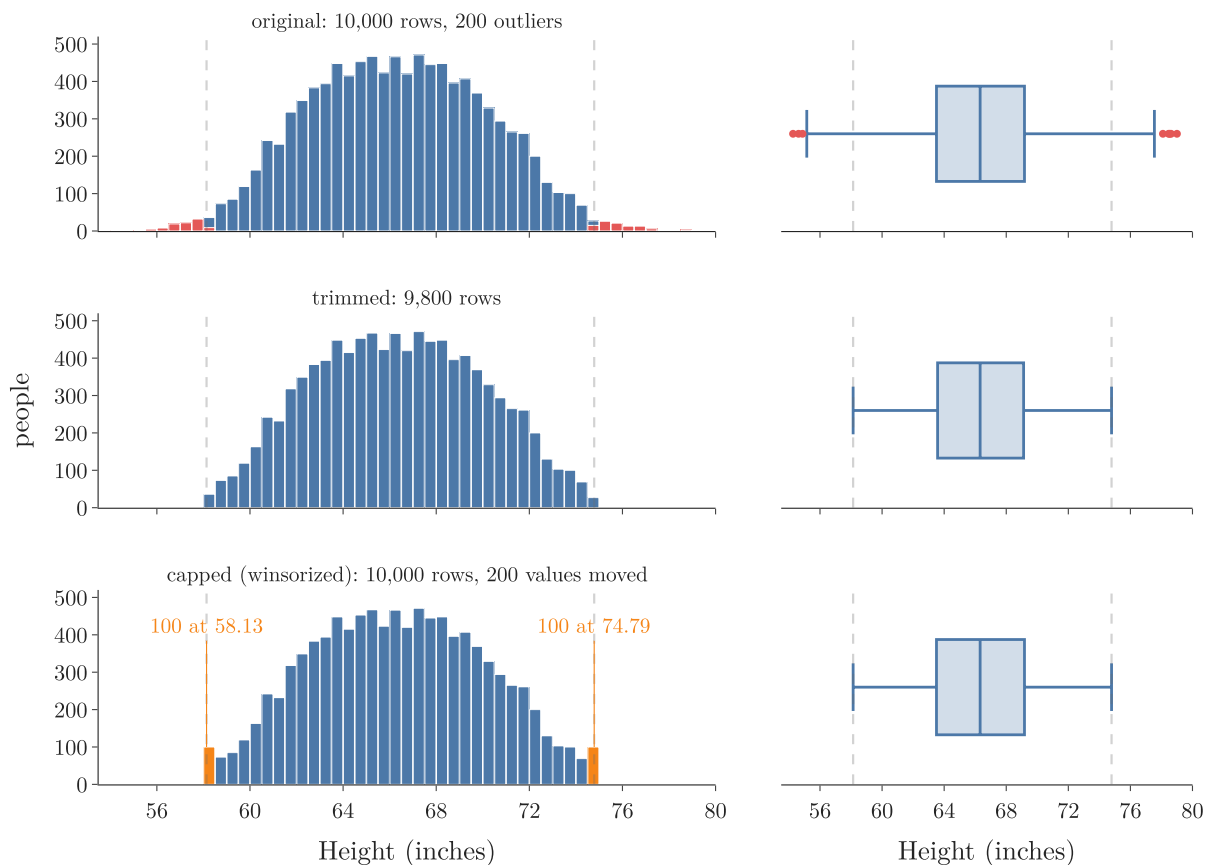


Figure 5: Before and after: the original column, the trimmed column (9,800 rows) and the capped column (10,000 rows, 200 values moved onto the limits); histograms on the left, box plots on the right, the limits dashed

Extra: Two shortcuts give the same kind of result.

- pandas: `clip` (G-70), as in `df["Height"].clip(lower_limit, upper_limit)`, gives exactly the same column as the two `np.where` calls.
- SciPy has a ready-made function, `scipy.stats.mstats.winsorize(values, limits=(0.01, 0.01))`. `winsorize` caps at the most extreme value that is kept (58.1345 and 74.7857), not at the interpolated percentile, so its limits differ in the fourth decimal. The `np.where` code is short enough that the extra library is not needed.

8. Choosing the cut-offs

Key point: Wider cut-offs, such as 0.5 and 99.5, remove less data; narrower ones, such as 5 and 95, remove more; we try a few and keep the one that gives the best results.

The percentiles decide how much data counts as an outlier, and the share is fixed by the choice itself:

Cut-offs	Lower limit	Upper limit	Outliers
0.5 and 99.5	57.31	75.69	100 (1%)
1 and 99	58.13	74.79	200 (2%)
2.5 and 97.5	59.26	73.70	500 (5%)
5 and 95	60.25	72.62	1,000 (10%)

Figure 6 slides the cut-offs from 0.5/99.5 to 5/95 on `Height`. Watch the red tails: the limits close in and the red share grows from 1% to 10%, whatever the shape of the bell.

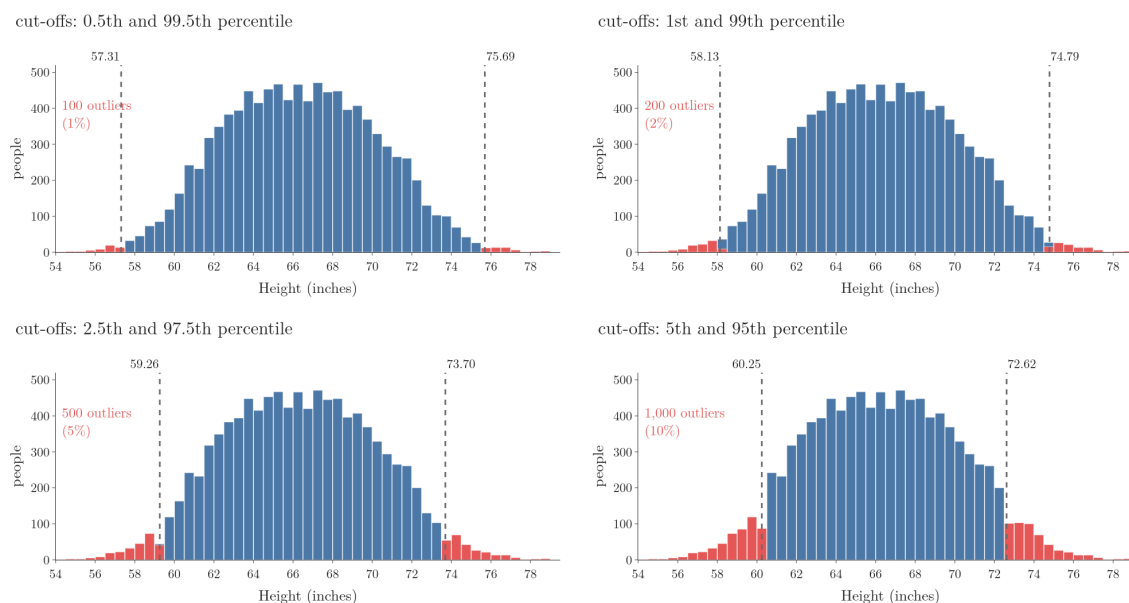


Figure 6: Sliding the cut-offs inwards on `Height`: the limits move in and the share flagged grows from 100 (1%) to 1,000 (10%)

The further out the cut-offs, the less data we trim or cap. A good habit is to start with wide cut-offs and only move them in if the model does better.

9. Learning the limits on the training set

Key point: Percentiles are learned from the data, so they should be learned on the training set only and then applied to the test set.

The limits follow the same train-only rule as the z-score limits of the [z-score Note](#) (section 11, learning the limits on the training set).

Extra: With an 80/20 split (`random_state=42`), the 8,000 training rows give limits of **58.16** and **74.83**:

- **Training set**, 8,000 rows: 80 below the lower limit, 80 above the upper limit.
- **Test set**, 2,000 rows: 23 below, 16 above.

On the training set the counts are exactly 1% at each end, by construction. On the test set they are only close to 1%, as expected from random sampling. Each test height has a 1% chance of falling below the training limit, so out of 2,000 we expect about 20, give or take about 4 (the standard deviation $\sqrt{2000 \times 0.01 \times 0.99} = 4.4$). The 23 and 16 we see are well within that. Trimming the training set leaves 7,840 rows; the Notebook shows both steps.

10. Strengths and limits of the percentile method

Key point: The method is simple and works on any shape, but it always flags the chosen share of values, even when the feature has no real outliers.

- **Simple:** two percentiles give both limits; one line of code each.
- **Shape-free:** the percentile method needs no bell shape and works on skewed features too.
- **Robust:** percentiles depend only on the order of the values, so extreme values cannot drag the limits out.

Extra: The catch is the fixed share. With the 1st and 99th percentiles, exactly 2% of the rows are flagged, whatever the data looks like. **Height** is almost normal: the z-score rule finds only 7 outliers in it and the IQR rule only 8, yet the percentile rule flags 200. **Weight**, with only 1 value beyond its box-plot fences, would also lose 200 values (last cell of the Notebook). So the percentile method decides *how many* values to treat, not *whether* a value is truly unusual.

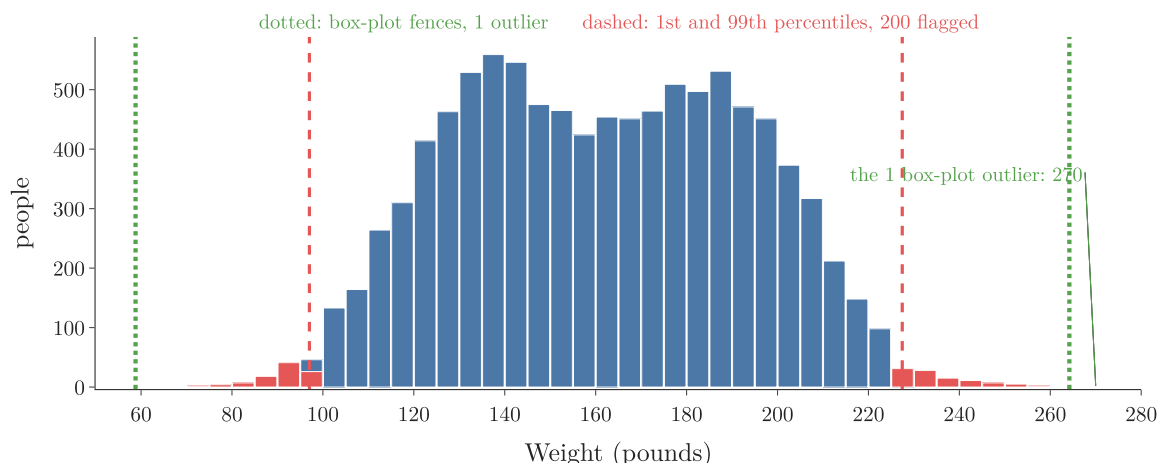


Figure 7: Weight: the box-plot fences (green, dotted) flag 1 value, at 270 pounds; the 1st and 99th percentiles (red, dashed) flag 200

In Figure 7, watch the gap between the red and the green lines: everything between them is a normal weight that the percentile rule still treats.

The intuition: winsorization calms the tails a little; it does not hunt for errors.

11. Comparing the three detection rules

Key point: Use the z-score rule for a normal feature, the IQR rule for a skewed one, and the percentile rule when we want to treat a fixed share of each tail in any feature.

This Note closes the outlier group. Figure 8 applies all three rules to the same **Height** feature. The z-score and IQR limits lie close together near the far ends; the percentile limits sit well inside them.

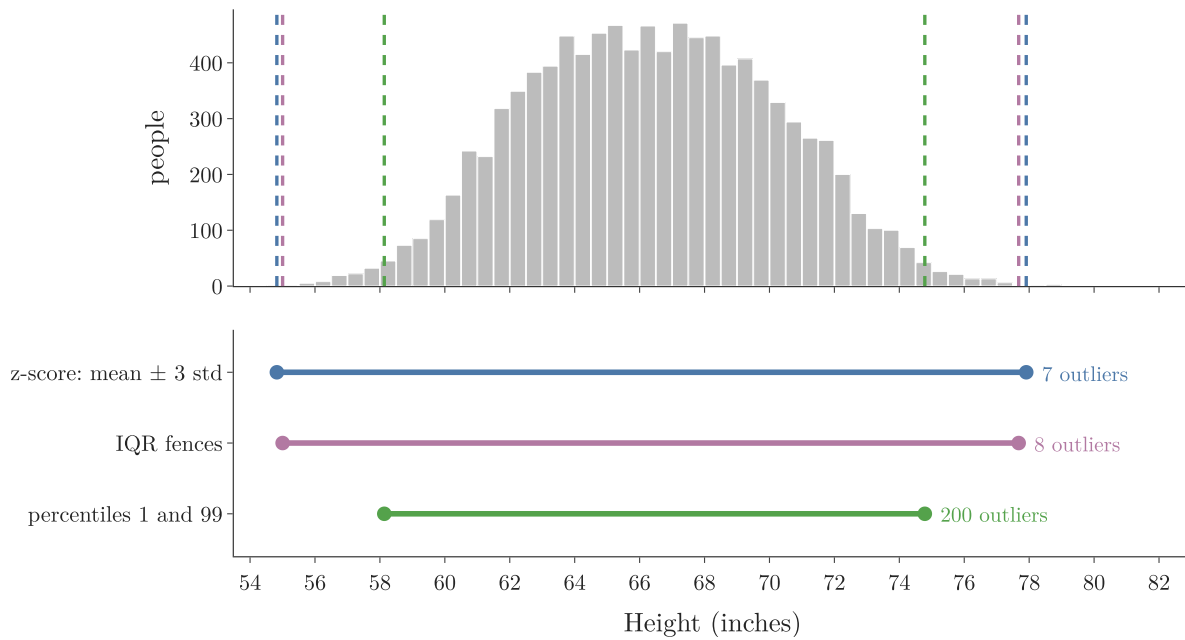


Figure 8: The three detection rules on Height: the z-score and IQR limits lie close to the ends and flag 7 and 8 values; the 1st and 99th percentiles lie further in and flag 200

	Z-score (Note 42)	IQR (Note 43)	Percentile (this Note)
Fits	roughly normal feature	skewed feature	any feature
Built on	mean and standard deviation	Q_1 and Q_3	two chosen percentiles
Limits	$\mu \pm 3\sigma$	$Q_1 - 1.5 \text{ IQR}$, $Q_3 + 1.5 \text{ IQR}$	P_1 , P_{99}
Pulled by outliers	yes	hardly	hardly
Share flagged	depends on the data	depends on the data	fixed by the cut-offs
Limits on Height	54.82 and 77.91	55.00 and 77.68	58.13 and 74.79
Outliers in Height	7	8	200

12. Summary

Step	What we do	On Height
Choose cut-offs	usually the 1st and 99th percentiles	1 and 99
Detect	lower = P_1 , upper = P_{99}	58.13 and 74.79; 100 below, 100 above
Trim	keep the rows inside the limits	9,800 rows left
Cap (winsorize)	move values beyond a limit onto it	10,000 rows; min 58.13, max 74.79
Better practice	learn the limits on the training set	58.16 and 74.83; 39 test values outside

- The percentile rule flags every value below a low percentile or above a high one; the cut-offs are our choice.
- Capping with percentile limits is called winsorization.
- The rule works on any shape, but always flags the same share of observations (2% with 1 and 99).
- Use the variables for the limits, never rounded numbers typed by hand.
- The limits should be learned on the training set only.

13. Sources

Built from

- CampusX, “Outlier Detection using the Percentile Method | Winsorization Technique”, YouTube, <https://www.youtube.com/watch?v=bcXA4CqRXvM>
- Khan Academy, “Calculating percentile”, YouTube, <https://www.youtube.com/watch?v=Ngvt8Q5tWk>

Other references

- pandas documentation, API reference, `pandas.Series.quantile`. pandas.pydata.org.
- Hyndman, R. J. and Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician*, 50(4), 361–365.
- Hastings, C., Mosteller, F., Tukey, J. W. and Winsor, C. P. (1947). Low moments for small samples: a comparative study of order statistics. *Annals of Mathematical Statistics*, 18(3), 413–426.
- Brillinger, D. R. (2002). John W. Tukey: his life and professional contributions. *Annals of Statistics*, 30(6), 1535–1575.

14. Key terms

Term	Meaning
Feature	An input variable: one column of the data table
Observation	One record: one row of the data table
Target	The output a model predicts
Percentile	The value below which a given share of the data lies
Percentile rank	The percentile at which a given value falls
Trimming	Removing the rows that hold outliers
Capping	Replacing every value beyond a limit with the limit itself
Percentile method	Outlier detection that flags values below a low percentile or above a high one (e.g. 1st and 99th); for any feature
Winsorization	Capping with limits set by percentiles: values beyond a limit are replaced with the limit
Cut-offs	The two percentiles chosen as limits, such as 1 and 99 or 5 and 95
Linear interpolation	Placing a percentile between two neighbouring sorted values, in proportion to its position; the pandas default
<code>np.where</code>	NumPy function that picks one value where a condition is true and another where it is false
<code>clip</code>	pandas method that moves every value below a lower bound up to it and every value above an upper bound down to it